

Raport

Przetwarzanie i analiza danych przestrzennych

Oracle spatial

Imiona i nazwiska:

Dawid Mularczyk

Michał Niezgoda

Celem ćwiczenia jest zapoznanie się ze sposobem przechowywania, przetwarzania i analizy danych przestrzennych w bazach danych (na przykładzie systemu Oracle spatial)

Swoje odpowiedzi wpisuj w miejsca oznaczone jako:

Wyniki, zrzut ekranu, komentarz

-- ...

Do wykonania ćwiczenia (zadania 1 – 6) i wizualizacji danych wykorzystaj Oracle SQL Developer. Alternatywnie możesz wykonać analizy w środowisku Python/Jupyter Notebook

Do wykonania zadania 7 wykorzystaj środowisko Python/Jupyter Notebook

Raport należy przestać w formacie pdf.

Należy też dołączyć raport zawierający kod w formacie źródłowym.

Np.

- plik tekstowy .sql z kodem poleceń
- plik .md zawierający kod wersji tekstowej
- notebook programu jupyter – plik .ipynb

Zamieść kod rozwiązania oraz zrzuty ekranu pokazujące wyniki, (dołącz kod rozwiązania w formie tekstowej/źródłowej)

Zwróć uwagę na formatowanie kodu

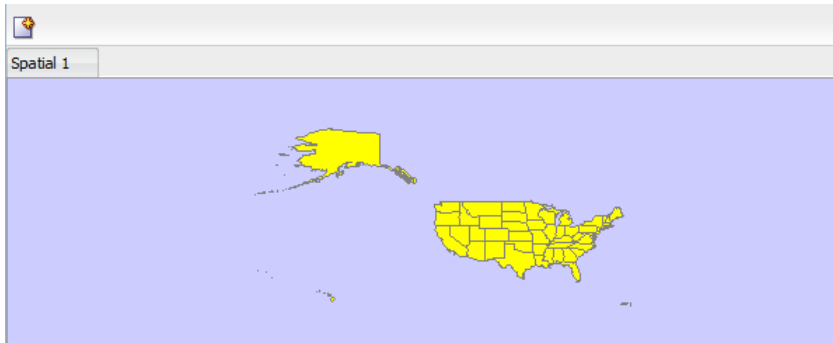
Zadanie 1

Zwizualizuj przykładowe dane

US_STATES

Wyniki, zrzut ekranu, komentarz

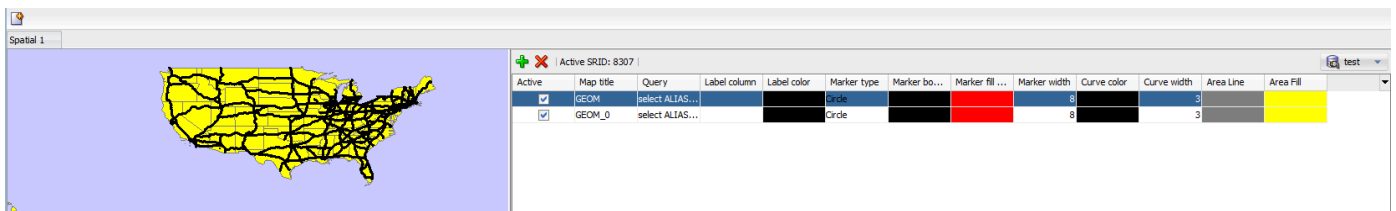
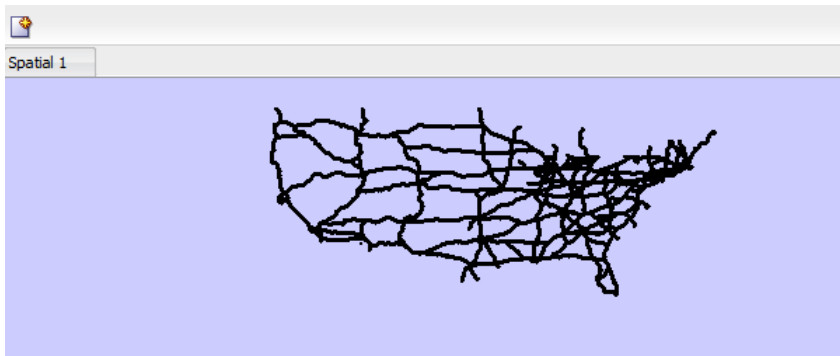
```
select * from us_states
```



US_INTERSTATES

Wyniki, zrzut ekranu, komentarz

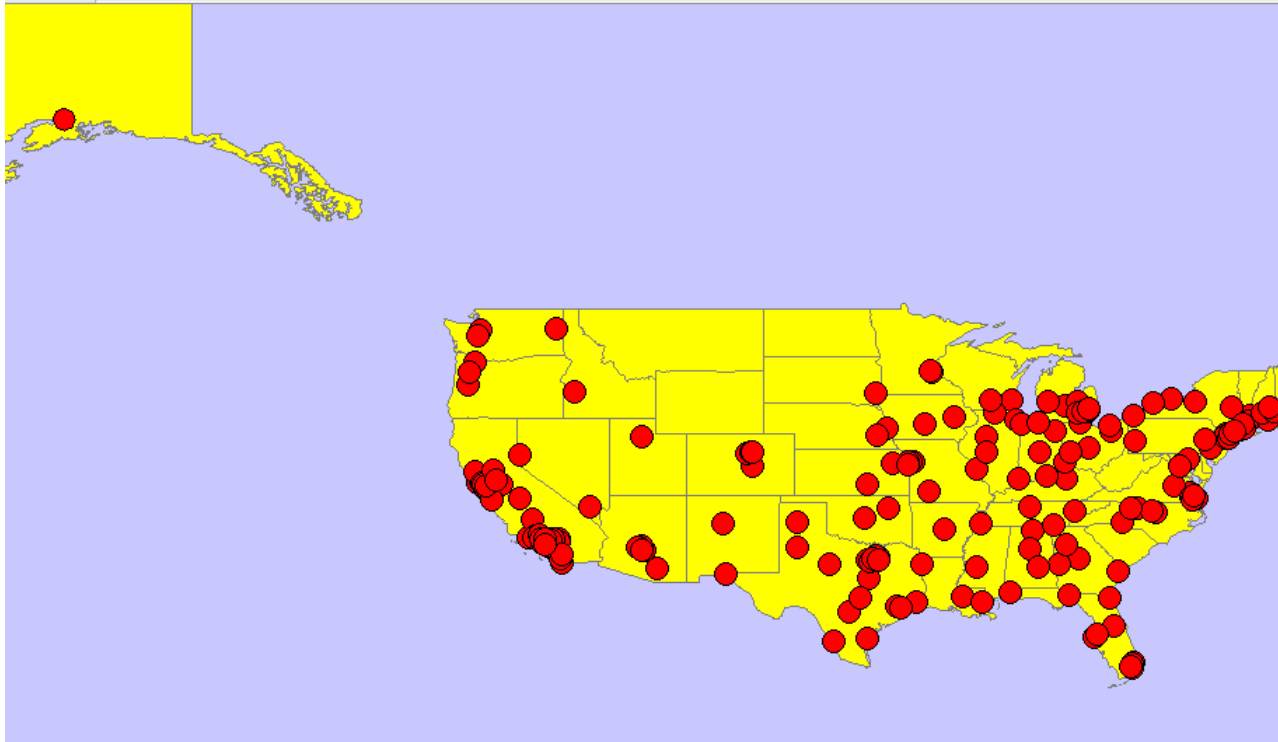
```
select * from us_interstates
```



US_CITIES

Wyniki, zrzut ekranu, komentarz

```
select * from us_cities
```



US_RIVERS

Wyniki, zrzut ekranu, komentarz

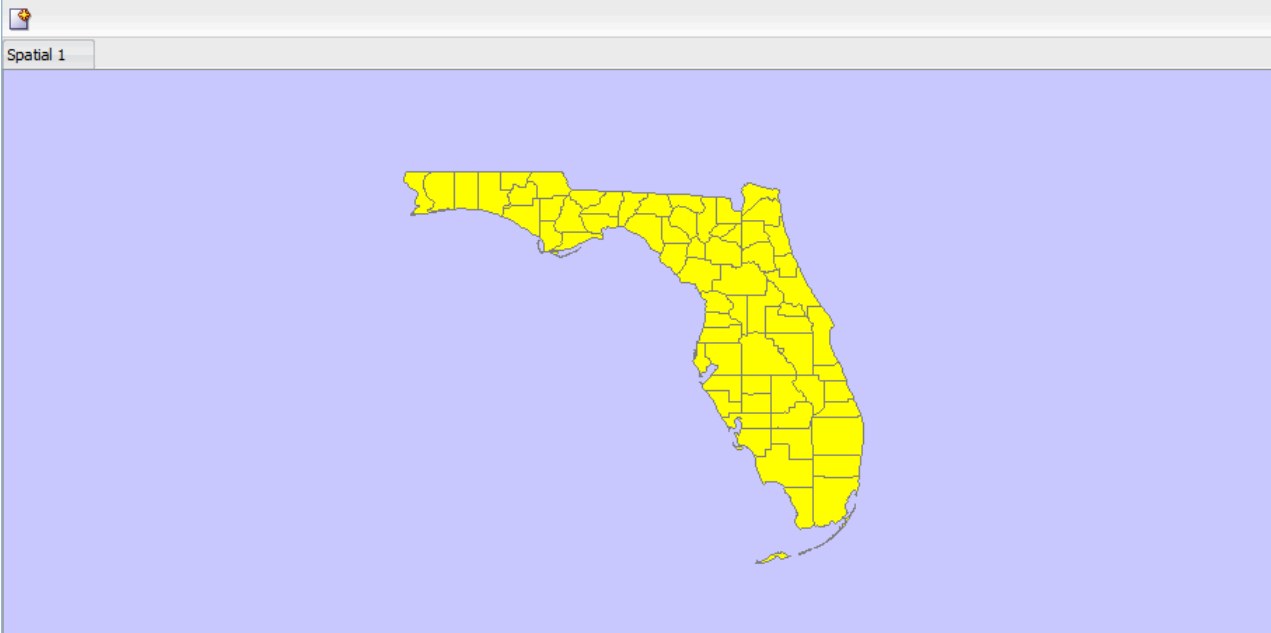
```
select * from us_rivers
```



US_COUNTIES

Wyniki, zrzut ekranu, komentarz

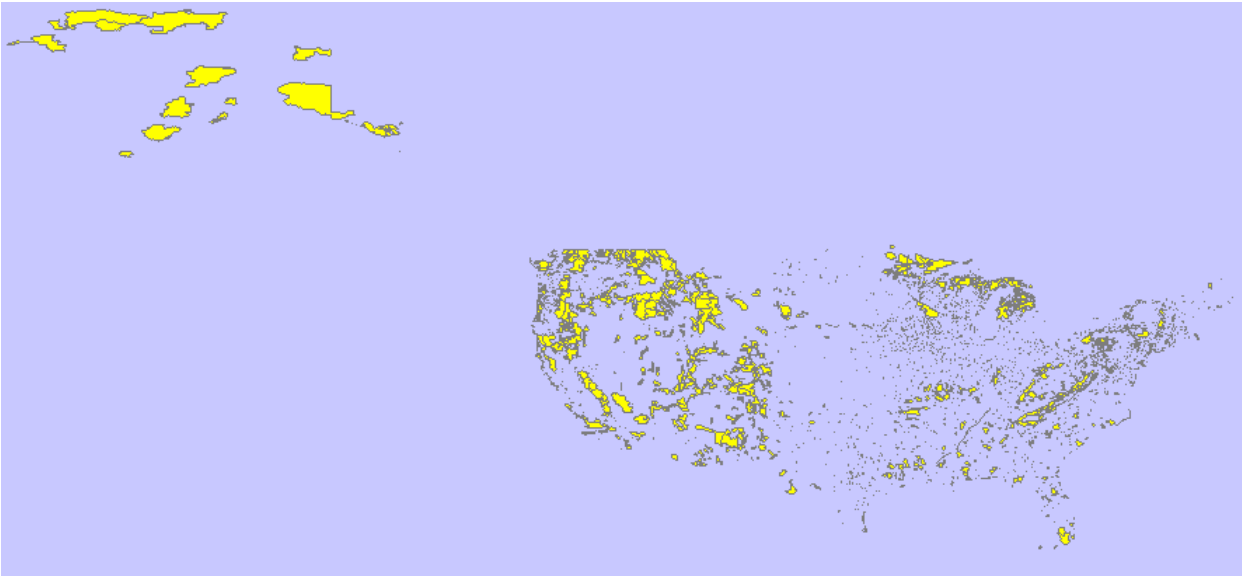
```
select * from us_counties where state_abrv = 'FL'
```



US_PARKS

Wyniki, zrzut ekranu, komentarz

```
select * from us_parks;
```



Zadanie 2

Znajdź wszystkie stany (us_states) których obszary mają część wspólną ze wskazaną geometrią (prostokątem)

Pokaż wynik na mapie.

prostokąt

```
SELECT sdo_geometry(2003, 8307, null,
  sdo_elem_info_array(1, 1003, 3),
  sdo_ordinate_array(-117.0, 40.0, -90., 44.0)) g
FROM dual;
```

Prostokąt o współrzędnych (-117.0, 40.0) – (-90.0, 44.0) w układzie WGS84 (SRID 8307), obejmujący obszar środkowo-zachodnich Stanów Zjednoczonych.

```
-- ...
```

Użyj funkcji SDO_FILTER

```
SELECT state, geom FROM us_states
WHERE sdo_filter(geom,
  sdo_geometry(2003, 8307, null,
    sdo_elem_info_array(1, 1003, 3),
    sdo_ordinate_array(-117.0, 40.0, -90., 44.0))
) = 'TRUE';
```

Zwróć uwagę na liczbę zwróconych wierszy (16)

SDO_FILTER zwróciło **16 wierszy**. Funkcja działa na podstawie bounding boxów (prostokątów otaczających geometrie), dlatego może zwracać fałszywe trafienia — zwraca stany, których bounding box nachodzi na prostokąt wyszukiwania, nawet jeśli sama geometria stanu go nie przecina.

```
-- ...
```

Użyj funkcji SDO_ANYINTERACT

```
SELECT state, geom FROM us_states
WHERE sdo_anyinteract(geom,
  sdo_geometry(2003, 8307, null,
    sdo_elem_info_array(1, 1003, 3),
    sdo_ordinate_array(-117.0, 40.0, -90., 44.0))
) = 'TRUE';
```

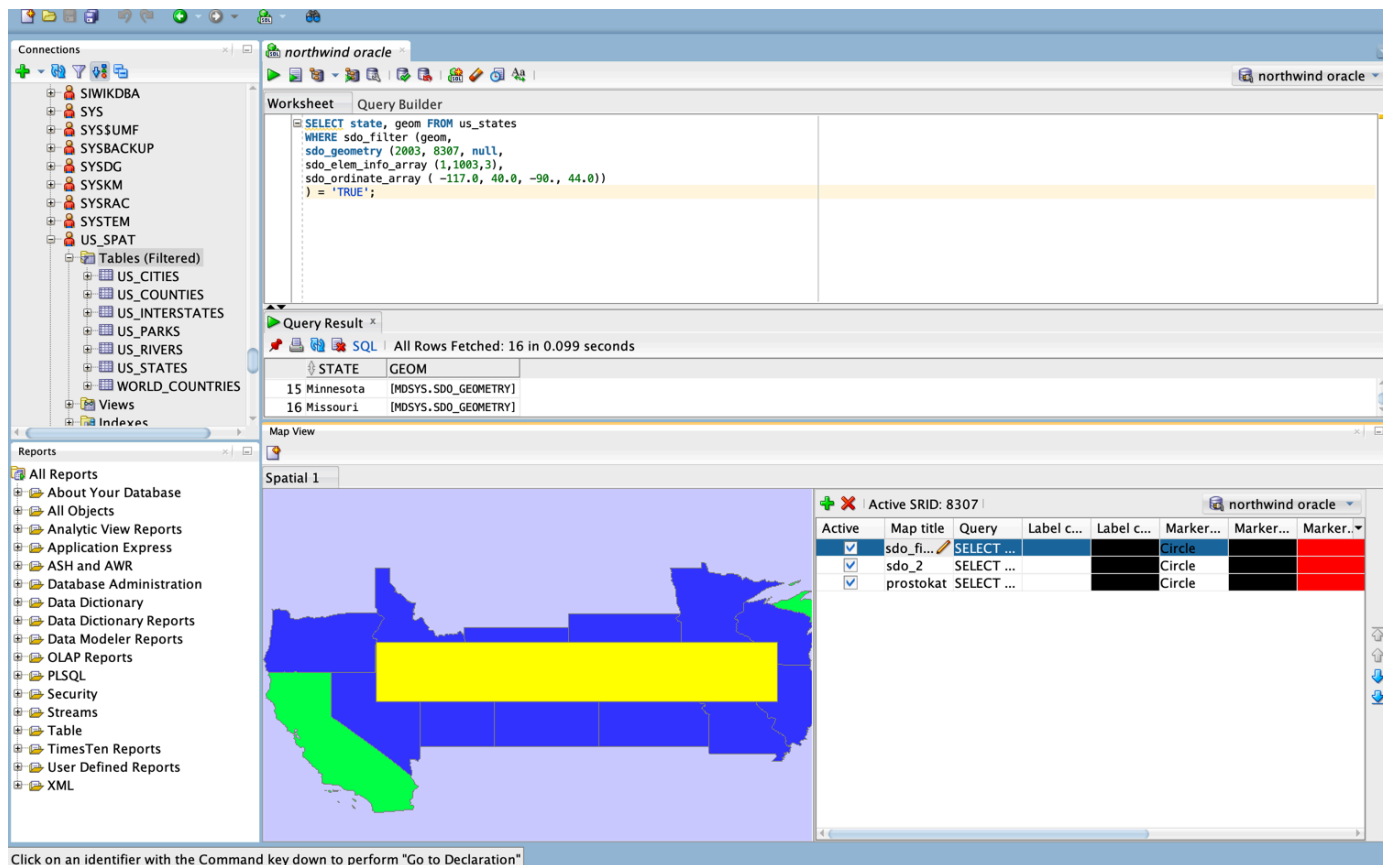
Porównaj wyniki sdo_filter i sdo_anyinteract

Pokaż wynik na mapie

SDO_ANYINTERACT zwróciło **14 wierszy** — o 2 mniej niż SDO_FILTER. Funkcja wykonuje dokładne obliczenie geometryczne i eliminuje fałszywe trafienia. Na mapie widać, że niektóre stany (np. Kalifornia) zostały zwrócone przez SDO_FILTER, ponieważ ich bounding box zachodzi na prostokąt wyszukiwania, ale geometria stanu faktycznie nie ma z nim części wspólnej — SDO_ANYINTERACT poprawnie je odrzuca.

Na poniższym zrzucie ekranu: **zielony** = wyniki SDO_FILTER (16 stanów), **niebieski** = wyniki SDO_ANYINTERACT (14 stanów), **żółty** = prostokąt wyszukiwania.

```
SELECT state, geom FROM us_states
WHERE sdo_anyinteract(geom,
  sdo_geometry(2003, 8307, null,
    sdo_elem_info_array(1, 1003, 3),
    sdo_ordinate_array(-117.0, 40.0, -90.0, 44.0))
) = 'TRUE';
```



Zadanie 3

0.5

Znajdź wszystkie parki (us_parks) których obszary znajdują się wewnątrz stanu Wyoming

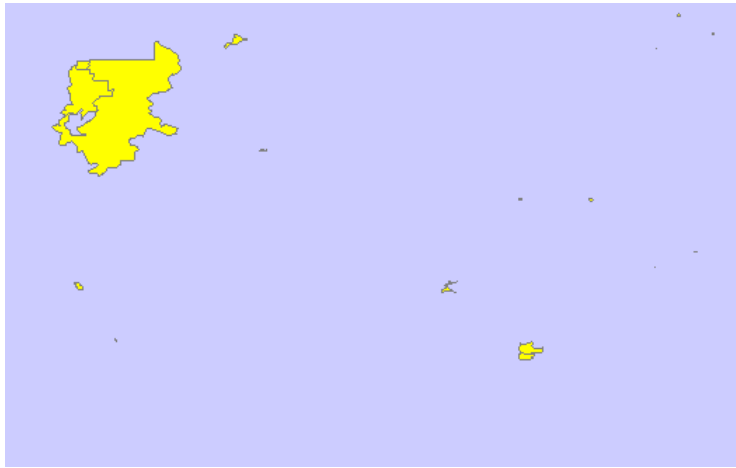
Użyj funkcji SDO_INSIDE

```
SELECT p.name, p.geom
FROM us_parks p, us_states s
WHERE s.state = 'Wyoming'
AND SDO_INSIDE (p.geom, s.geom) = 'TRUE';
```

W przypadku wykorzystywania narzędzia SQL Developer, w celu wizualizacji na mapie użyj podzapytania

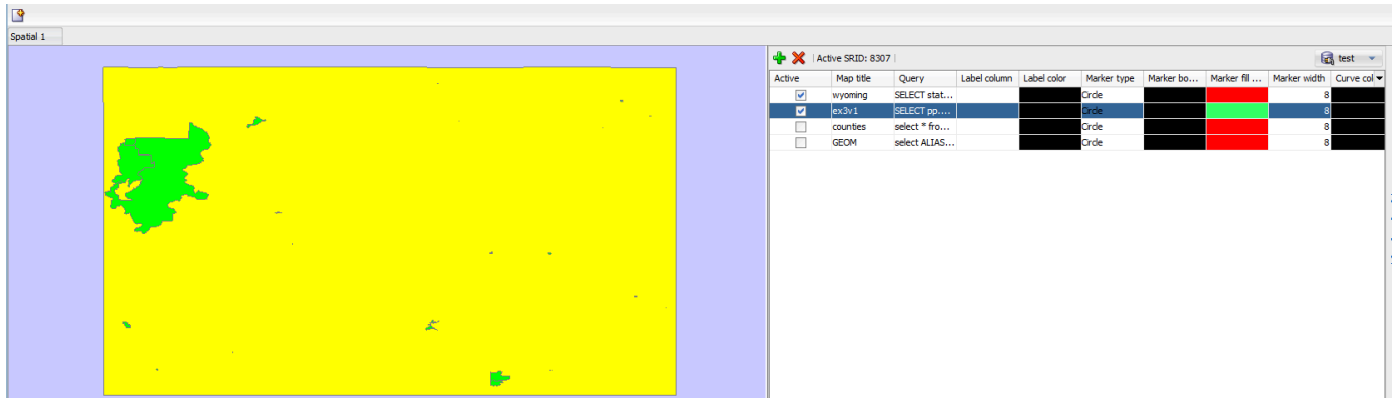
```
SELECT pp.name, pp.geom FROM us_parks pp
WHERE id IN
(
    SELECT p.id
    FROM us_parks p, us_states s
    WHERE s.state = 'Wyoming'
    AND SDO_INSIDE (p.geom, s.geom) = 'TRUE'
)
```

Wyniki, zrzut ekranu, komentarz



```
SELECT state, geom FROM us_states
WHERE state = 'Wyoming'
```

Wyniki, zrzut ekranu, komentarz



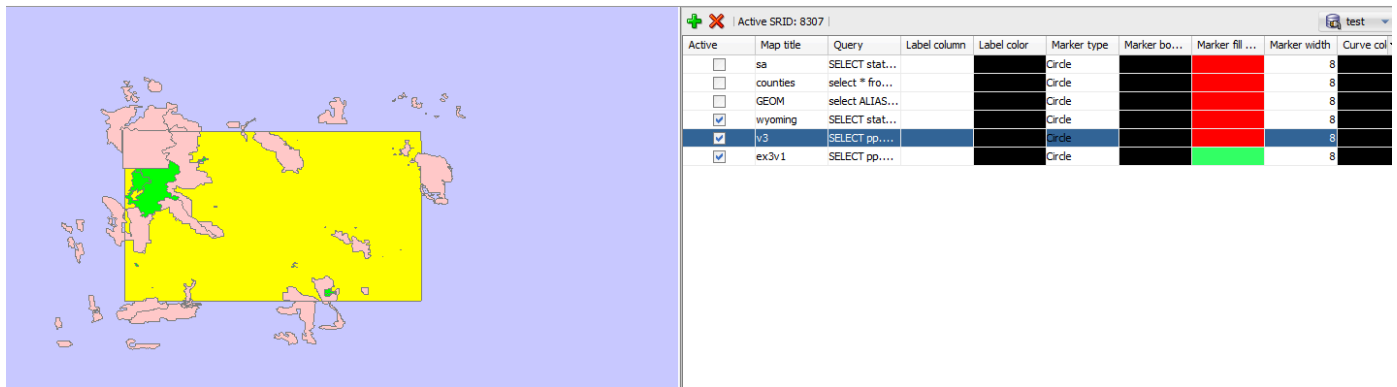
Porównaj wynik z:

```
SELECT p.name, p.geom
FROM us_parks p, us_states s
WHERE s.state = 'Wyoming'
AND SDO_ANYINTERACT (p.geom, s.geom ) = 'TRUE';
```

W celu wizualizacji użyj podzapytania

Wyniki, zrzut ekranu, komentarz

```
SELECT pp.name, pp.geom FROM us_parks pp
WHERE id IN
(
  SELECT p.id
  FROM us_parks p, us_states s
  WHERE s.state = 'Wyoming'
  AND SDO_ANYINTERACT (p.geom, s.geom) = 'TRUE'
)
```



Różnica między zapytaniem:

SDO_INSIDE - zwraca tylko parki które są w całości wewnątrz stanu Wyoming, granice się nie stykają

SDO_ANYINTERACT - zwraca parki, które mają cokolwiek wspólnego ze stanem Wyoming, czyli też te które leżą na granicy lub ją przekraczają

0.6

Zadanie 4

Znajdź wszystkie jednostki administracyjne (us_counties) wewnątrz stanu New Hampshire

```
SELECT c.county, c.state_abrv, c.geom
FROM us_counties c, us_states s
WHERE s.state = 'New Hampshire'
AND SDO_RELATE ( c.geom,s.geom, 'mask=INSIDE+COVEREDBY') = 'TRUE';

SELECT c.county, c.state_abrv, c.geom
FROM us_counties c, us_states s
WHERE s.state = 'New Hampshire'
AND SDO_RELATE ( c.geom,s.geom, 'mask=INSIDE') = 'TRUE';

SELECT c.county, c.state_abrv, c.geom
FROM us_counties c, us_states s
WHERE s.state = 'New Hampshire'
AND SDO_RELATE ( c.geom,s.geom, 'mask=COVEREDBY') = 'TRUE';
```

W przypadku wykorzystywania narzędzia SQL Developer, w celu wizualizacji danych na mapie należy użyć podzapytania z **EXISTS** (zamiast JOIN na zewnętrznym poziomie):

```
-- INSIDE+COVEREDBY
SELECT cc.county, cc.state_abrv, cc.geom
FROM us_counties cc
WHERE EXISTS (
  SELECT 1 FROM us_states s
  WHERE s.state = 'New Hampshire'
  AND SDO_RELATE(cc.geom, s.geom, 'mask=INSIDE+COVEREDBY') = 'TRUE'
);

-- INSIDE
SELECT cc.county, cc.state_abrv, cc.geom
FROM us_counties cc
WHERE EXISTS (
  SELECT 1 FROM us_states s
  WHERE s.state = 'New Hampshire'
  AND SDO_RELATE(cc.geom, s.geom, 'mask=INSIDE') = 'TRUE'
);

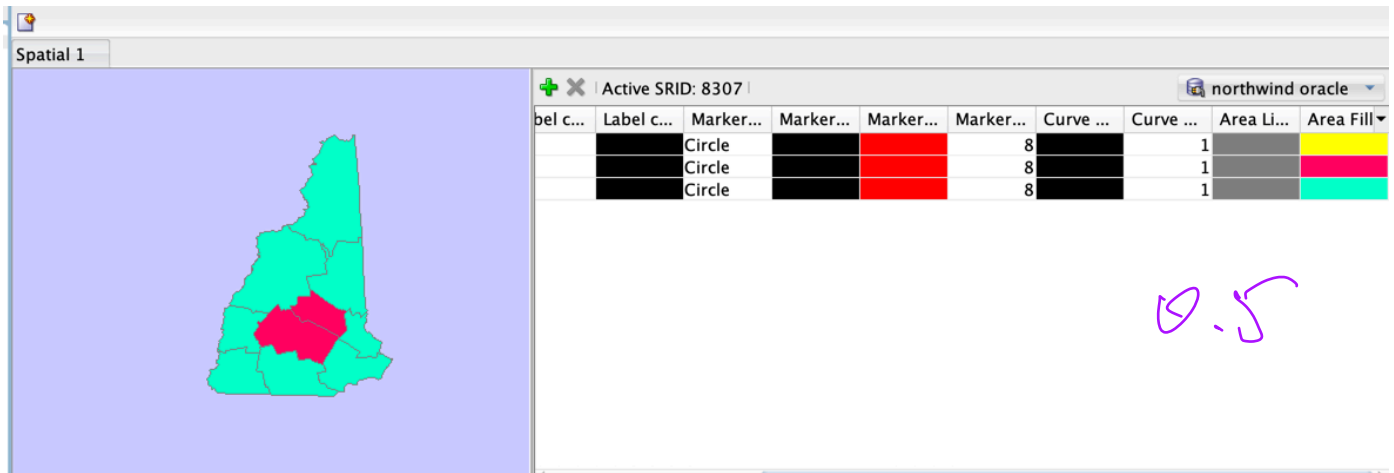
-- COVEREDBY
SELECT cc.county, cc.state_abrv, cc.geom
FROM us_counties cc
WHERE EXISTS (
  SELECT 1 FROM us_states s
  WHERE s.state = 'New Hampshire'
  AND SDO_RELATE(cc.geom, s.geom, 'mask=COVEREDBY') = 'TRUE'
);
```

INSIDE+COVEREDBY zwróciło **10 wierszy** — wszystkie powiaty stanu New Hampshire.

INSIDE zwróciło **2 wiersze**: Merrimack i Belknap — jedyne dwa powiaty, których granice nie stykają się z granicą stanu NH (leżą w całości wewnątrz).

COVEREDBY zwróciło **8 wierszy**: Cheshire, Hillsborough, Sullivan, Rockingham, Strafford, Grafton, Carroll, Coos — powiaty, których granica pokrywa się z granicą stanu.

Maska **INSIDE** i **COVEREDBY** są rozłączne i sumują się do **INSIDE+COVEREDBY** ($2 + 8 = 10$). Na mapie warstwy COVEREDBY i INSIDE+COVEREDBY wyglądają podobnie, ponieważ różnią się tylko o 2 centralne powiaty (widoczne jako osobna warstwa INSIDE).



Zadanie 5

Znajdź wszystkie miasta w odległości 50 mili od drogi (us_interstates) I4

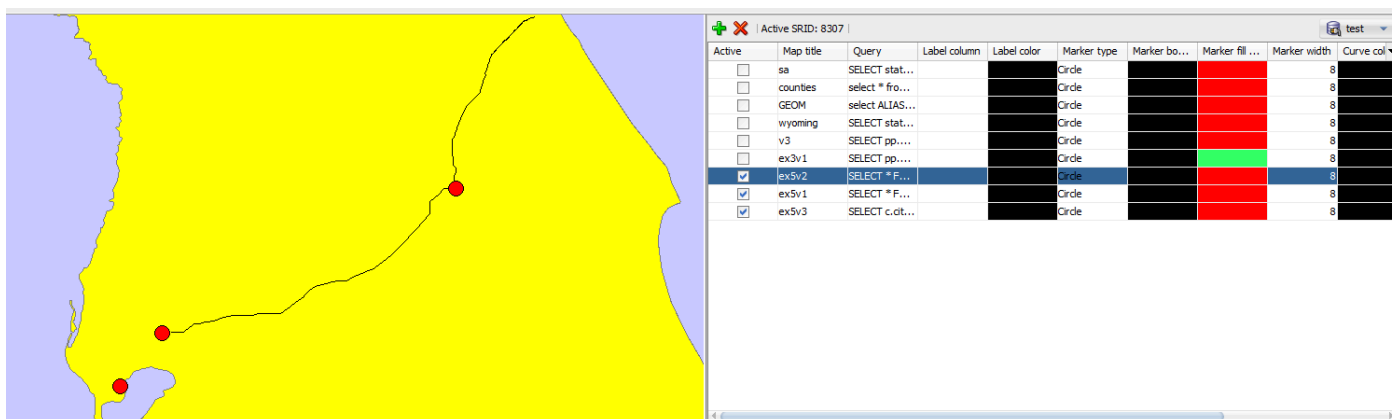
Pokaż wyniki na mapie

```
SELECT * FROM us_interstates
WHERE interstate = 'I4'

SELECT * FROM us_states
WHERE state_abrv = 'FL'

SELECT c.city, c.state_abrv, c.location
FROM us_cities c
WHERE ROWID IN
(
SELECT c.rowid
FROM us_interstates i, us_cities c
WHERE i.interstate = 'I4'
AND sdo_within_distance (c.location, i.geom, 'distance=50 unit=mile') = 'TRUE'
)
```

Wyniki, zrzut ekranu, komentarz



Dodatkowo:

- Znajdź wszystkie drogi które przecinają rzekę Mississippi
- Znajdź wszystkie miasta w odległości od 15 do 30 mil od drogi 'I275'

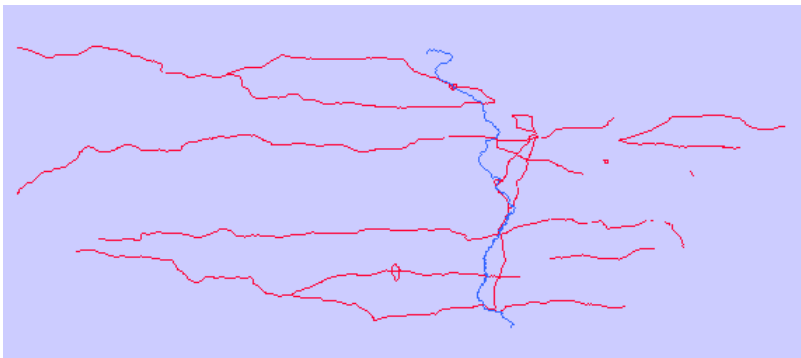
c) ltp. (własne przykłady)

Wyniki, zrzut ekranu, komentarz (dla każdego z podpunktów)

a) **Znajdź wszystkie drogi które przecinają rzekę Mississippi**

```
-- podpunkt a
-- sama rzeka
SELECT name, geom
FROM us_rivers
WHERE name = 'Mississippi'

SELECT i.interstate, i.geom
FROM us_interstates i
WHERE i.interstate IN
(
    SELECT i2.interstate
    FROM us_interstates i2, us_rivers r
    WHERE r.name = 'Mississippi'
    AND SDO_ANYINTERACT (i2.geom, r.geom) = 'TRUE'
)
```

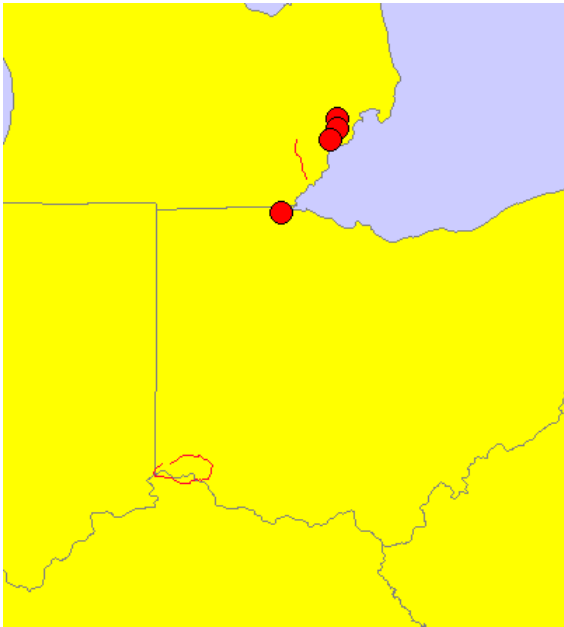


b) **Znajdź wszystkie miasta w odległości od 15 do 30 mil od drogi 'I275'**

```
SELECT interstate, geom
FROM us_interstates
WHERE interstate = 'I275'

-- miasta w odległości 15-30 mil od I275
SELECT c.city, c.state_abrv, c.location
FROM us_cities c
WHERE ROWID IN
(
    SELECT c.rowid
    FROM us_interstates i, us_cities c
    WHERE i.interstate = 'I275'
    AND sdo_within_distance(
        c.location, i.geom,
        'distance=30 unit=mile'
    ) = 'TRUE'
    AND sdo_geom.sdo_distance(
        c.location,
        i.geom, 0.5, 'unit=mile'
    ) >= 15
)

-- tło
SELECT state, geom
FROM us_states
```

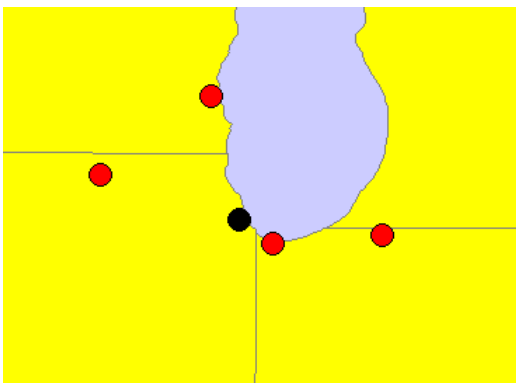


Przykłady własne

Przykład miasta w odległości 100 mil od Chicago

```
-- punkt na mapie Chicago
SELECT city, location
FROM us_cities
WHERE city = 'Chicago'

-- w odległości 100 mil
SELECT c1.city, c1.state_abrv, c1.location
FROM us_cities c1
WHERE c1.city != 'Chicago'
AND ROWID IN
(
  SELECT c1.rowid
  FROM us_cities c1, us_cities c2
  WHERE c2.city = 'Chicago'
  AND sdo_within_distance(
    c1.location, c2.location,
    'distance=100 unit=mile'
  ) = 'TRUE'
)
```



0.6

Zadanie 6

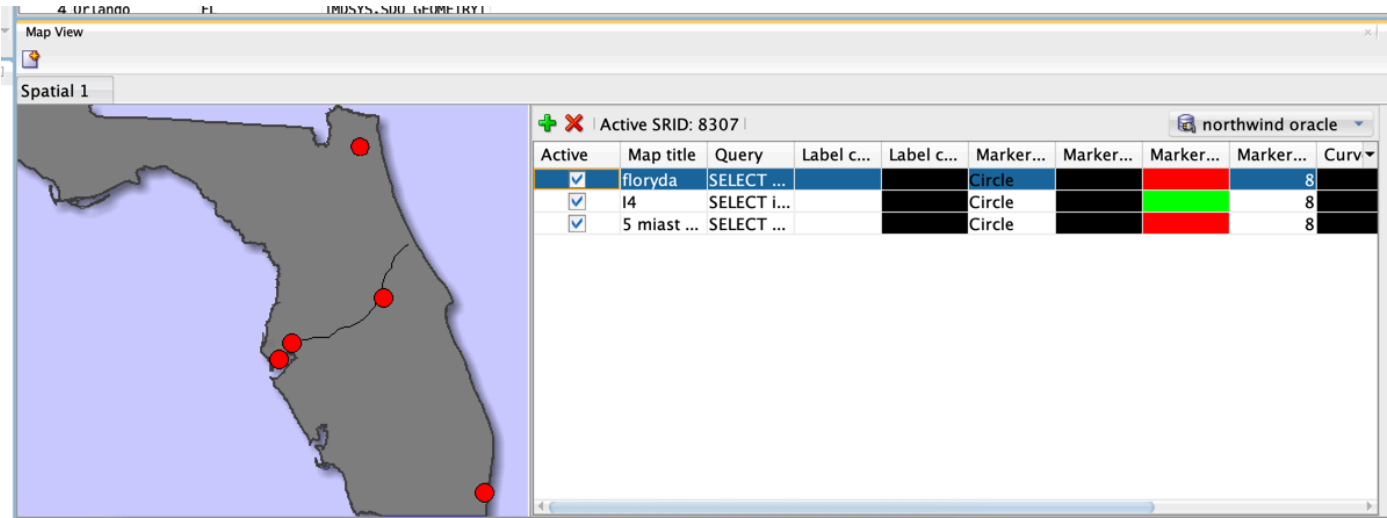
Znajdz 5 miast najbliższych drogi I4

Zapytanie wykonane z użyciem scalar subquery zamiast JOIN, aby umożliwić wizualizację w Map View (outer query z jednej tabeli).

```
SELECT cc.city, cc.state_abrv, cc.location
FROM us_cities cc
```

```
WHERE sdo_nn(
  cc.location,
  (SELECT geom FROM us_interstates WHERE interstate = 'I4'),
  'sdo_num_res=5'
) = 'TRUE';
```

Wynik — 5 miast najbliższych drogi I4 (Floryda): Tampa, Jacksonville, St Petersburg, Orlando, Fort Lauderdale



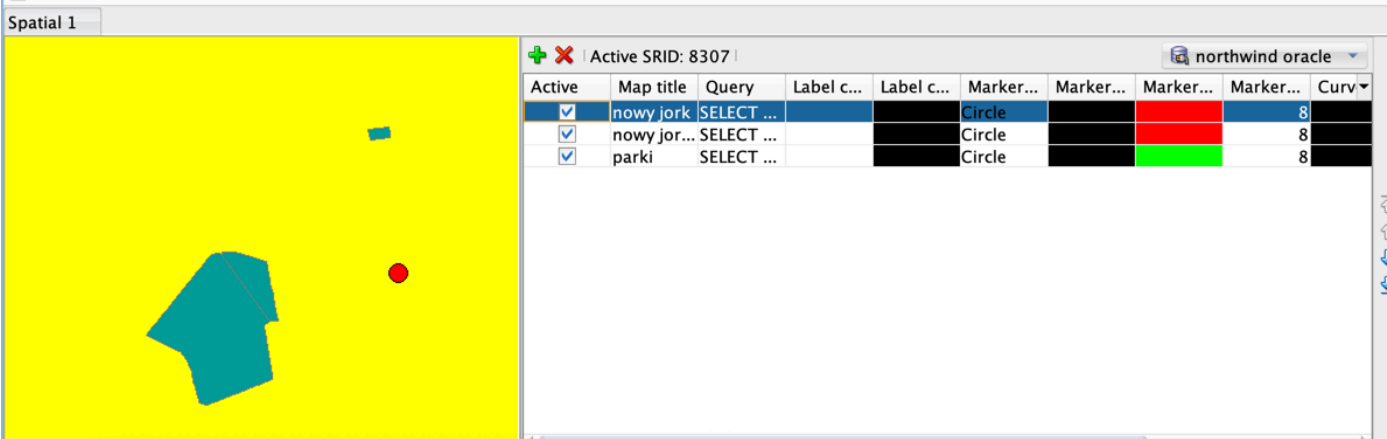
Dodatkowo:

a) Podaj 3 parki narodowe do których jest najbliżej z Nowego Jorku, oblicz odległości do tych parków

```
SELECT pp.name, pp.geom,
       sdo_nn_distance(1) AS distance_miles
FROM us_parks pp
WHERE sdo_nn(
  pp.geom,
  (SELECT location FROM us_cities WHERE city = 'New York'),
  'sdo_num_res=3 unit=mile', 1
) = 'TRUE'
ORDER BY distance_miles;
```

Wynik — 3 parki najbliżej Nowego Jorku:

Park	Odległość
Institute Park	1.0 mil
Prospect Park	1.1 mil
Thompkins Park	1.3 mil

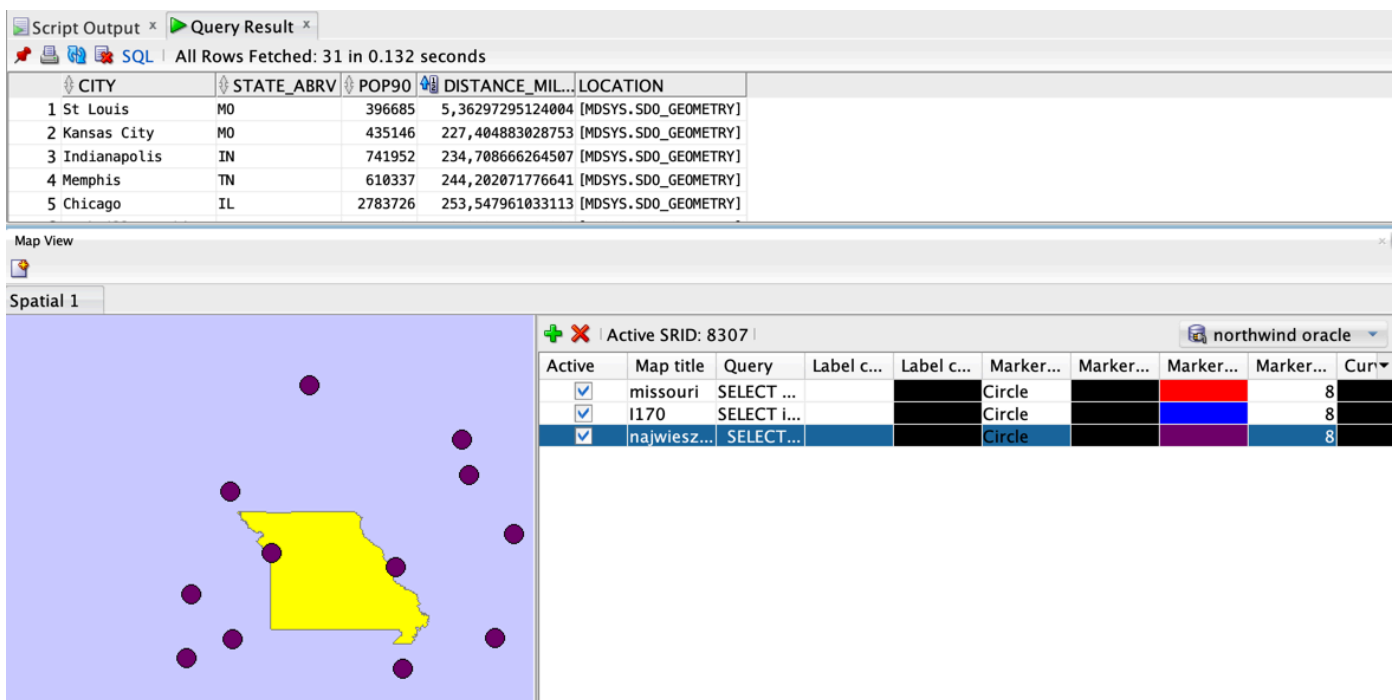


b) Znajdz 5 najbliższych dużych miast (o populacji powyżej 300 tys) od drogi 'I170'

```
SELECT cc.city, cc.state_abrv, cc.pop90,
       sdo_nn_distance(1) AS distance_miles,
       cc.location
FROM us_cities cc
WHERE cc.pop90 > 300000
AND sdo_nn(
    cc.location,
    (SELECT geom FROM us_interstates WHERE interstate = 'I170'),
    'sdo_num_res=100 unit=mile', 1
) = 'TRUE';
```

Uwaga dot. ograniczenia do 5 wyników: Operator `SDO_NN` z parametrem `sdo_num_res=5` zwraca tylko 1 miasto, ponieważ Oracle najpierw wyszukuje 5 geometrycznie najbliższych miast (niezależnie od populacji), a dopiero potem aplikuje filtr `pop90 > 300000` — spośród 5 najbliższych miast tylko St. Louis spełnia warunek populacji. Parametr `sdo_batch_size` który powinien rozwiązać ten problem, nie działa ze scalar subquery w tej wersji Oracle. Zagnieżdżone query z `ORDER BY + ROWNUM <= 5` daje poprawne 5 wyników, ale uniemożliwia wizualizację w Map View (ORA-01446). Zdecydowano o użyciu `sdo_num_res=100` — zapytanie zwraca wszystkie duże miasta w zasięgu, a 5 najbliższych (posortowanych po odległości) wskazano poniżej.

Miasto	Stan	Populacja	Odległość
St Louis	MO	396 685	5.4 mil
Kansas City	MO	435 146	227.4 mil
Indianapolis	IN	741 952	234.7 mil
Memphis	TN	610 337	244.2 mil
Chicago	IL	2 783 726	253.5 mil

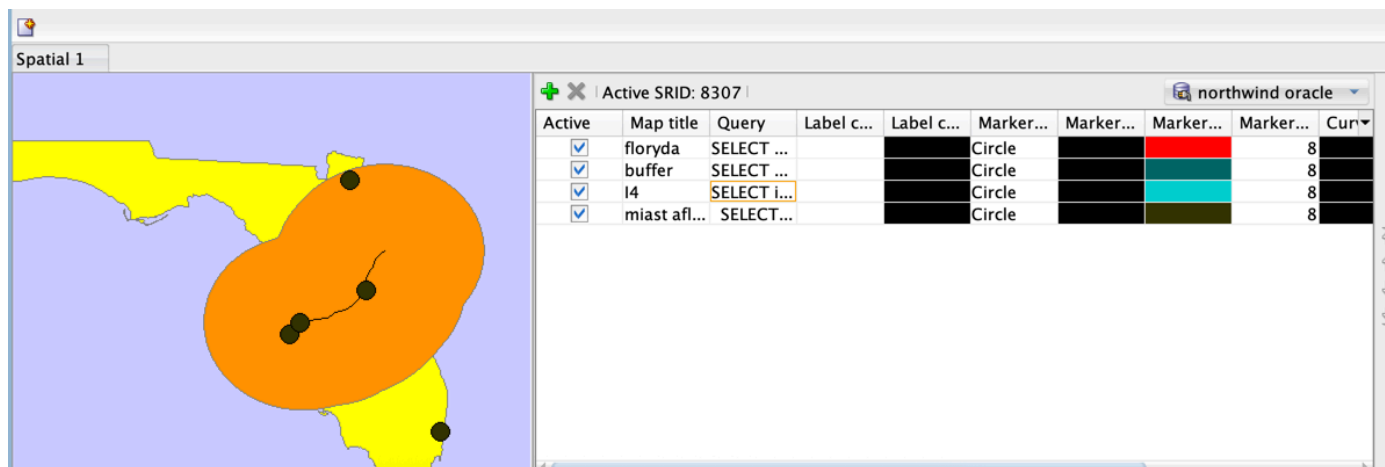


c) Własne przykłady

SDO_BUFFER — strefa 100 mil wokół drogi I4

```
SELECT SDO_GEOM.SDO_BUFFER(geom, 100, 0.5, 'unit=mile') AS geom
FROM us_interstates
WHERE interstate = 'I4';
```

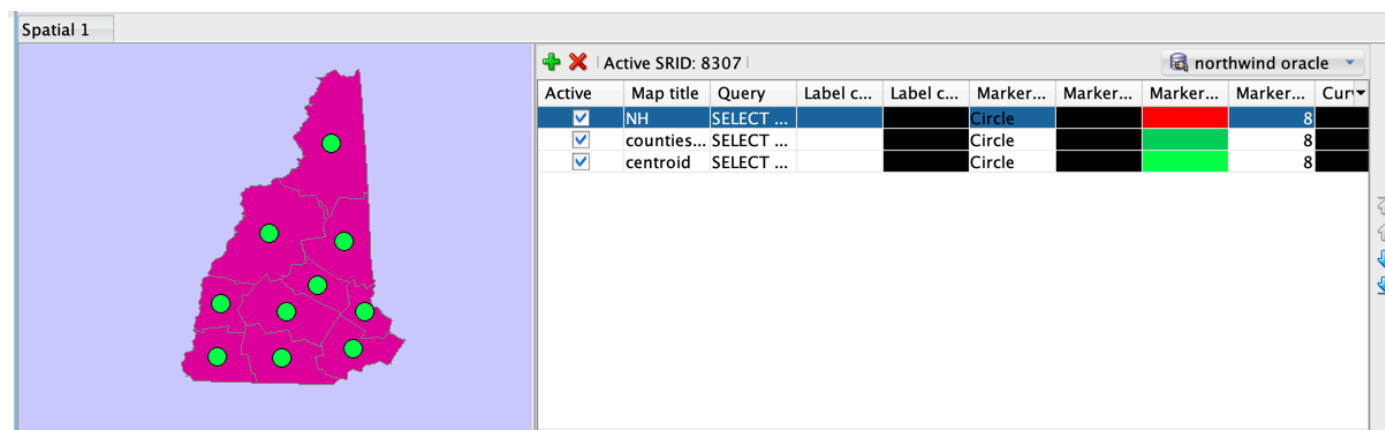
`SDO_BUFFER` tworzy wielokąt reprezentujący obszar w zadanej odległości od obiektu. Wynik to strefa 100 mil wokół drogi I4 — na mapie widoczna jako pomarańczowy wielokąt. Punkty to 5 najbliższych miast z zapytania `SDO_NN sdo_num_res=5`. Widać że jedno miasto (Fort Lauderdale, na południu) leży **poza** bufferem — `SDO_NN` zwraca N najbliższych niezależnie od odległości, podczas gdy buffer pokazuje dokładną strefę 100 mil. To dobrze ilustruje różnicę między `SDO_NN` (N najbliższych) a `SDO_WITHIN_DISTANCE` (wszystkie w promieniu).



SDO_CENTROID — środki geometryczne counties stanu New Hampshire

```
SELECT county, SDO_GEOM.SDO_CENTROID(geom, 0.5) AS centroid
FROM us_counties
WHERE state_abrv = 'NH';
```

SDO_CENTROID zwraca punkt będący geometrycznym środkiem ciężkości wielokąta. Dla każdego z 10 powiatów New Hampshire wyznaczono punkt centralny (zielone kropki) — nałożone na granice counties widać że centroid leży wewnątrz każdego powiatu. Przydatne np. do rozmieszczania etykiet na mapie lub obliczania odległości między powiatami.



Zadanie 7

2p

W notebooku

Punktacja

zad	pkt
1	0,5
2	0,5
3	0,5
4	0,5
5	1
6	2
7	2
razem	7

→ a dlaczego nie to

1p

S.T